PROJECT REPORT

Picturehouse Jaipur Project Report

Design and implementation plan for movie booking with shared rides

Jaipur, Rajasthan | Version 1.0 | 18 October 2026

Executive summary

Picturehouse Jaipur is a Jaipur-focused web product concept that connects cinema discovery and seat booking with optional ride sharing for moviegoers attending the same show. The workspace now contains a responsive clickable frontend prototype and a Java Spring Boot domain foundation. The external transaction providers and production database deployment are intentionally integration points; the prototype does not create real bookings or send real OTPs, payments, tickets or WhatsApp messages.

1. Problem and objectives

Moviegoers commonly plan tickets, transport and coordination in separate apps. Theatre owners also need a controlled way to publish listings and showtimes. This project brings those tasks into one Jaipur-first flow while preserving clear role boundaries and strong seat-allocation rules.

- Make nearby films and showtimes easy to compare.
- Prevent duplicate seat sales while keeping seat maps fresh across clients.
- Issue a verifiable QR ticket only after payment confirmation.
- Let customers coordinate a shared ride around the same screening.
- Demonstrate object-oriented design through cohesive domain objects, encapsulated state changes, interfaces and role-specific operations.

2. Essential feature operation

Feature	How it works
Signup and verification	First-time customers create an account, verify a mobile number and email, and obtain the customer role. Owner accounts are separately approved and assigned to one or more theatres.
Discovery	Customer chooses a Jaipur date, film or neighbourhood; the catalogue returns matching active shows with language, certification, format and venue details.
Live seat selection	A client reads the show inventory and subscribes to its seat-update topic. Selecting a seat requests a short hold from the API. A transaction with a database uniqueness/locking rule decides the winner; after commit the API broadcasts state.
Pricing and payment	Server pricing maps front/middle/recliner categories to ₹250/₹300/₹500 seed prices. The checkout request uses integer paise and INR. A signed, idempotent gateway webhook confirms the booking.

Feature	How it works
QR ticket and notifications	After confirmation, the server creates an opaque signed entry token, prepares a QR image and queues email and approved WhatsApp deliveries. Staff validation checks the show window and consumes the token once.
Ride share	A customer posts a ride tied to a show. Other authenticated customers ask to join and use access-controlled chat. Acceptance reserves a ride seat; the fare contribution is disclosed before confirmation.
Owner portal	Owner-scoped endpoints allow authorized changes to theatre, screen, film and showtime records. Aggregate occupancy is visible without exposing customer payment or contact details.
Theme	The frontend applies a light or dark design token set and stores the choice on the device.

3. Frontend, backend and database

Frontend

The user interface is built with React and Vite. It is a responsive single-page application with movie cards, theatre showtimes, a seat picker, checkout and dual-channel OTP dialogs, customer navigation, ride listings/chat and an owner view. Current interactions use local sample data and are not connected to the Java API. CSS design tokens support light/dark themes.

Backend

The backend uses Java 21 and Spring Boot. Domain classes such as `Movie`, `Theatre`, `Showtime`, `Booking`, `CarpoolRide` and `Account` model business concepts. Enums constrain roles, booking states and seat categories. Domain methods encapsulate state transitions and fare values. `PaymentGateway`, `OtpGateway` and `TicketDeliveryGateway` are interfaces that allow provider implementations to be substituted without coupling business rules to a vendor. Spring WebSocket configuration provides the transport foundation for live inventory events.

Database

PostgreSQL is the intended durable database, with H2 configured for local startup. Relational tables separate accounts, theatre ownership, venues, screens, movies, showtimes, physical seats, temporary holds, bookings, booking-seat snapshots, payment attempts, ticket credentials, ride posts, participants and chat messages. Amounts are integer paise; timestamps are stored in UTC and rendered in Jaipur local time. Database constraints and transaction isolation are essential for seat and ride-seat concurrency.

4. Object-oriented programming demonstration

OOP concept	Project example
Encapsulation	`Booking.confirm()` protects allowed booking transitions; `CarpoolRide.reserveSeat()` prevents overselling available places; fields are private.
Abstraction	Provider interfaces define payment, OTP and ticket delivery operations without exposing vendor details to booking logic.
Polymorphism	Different gateway implementations can satisfy one interface: sandbox, Razorpay, email provider or WhatsApp provider can be selected by configuration.

OOP concept	Project example
Inheritance and composition	JPA entities share identity through persistence conventions; richer design should favor composition for account roles, theatre membership and notification preferences. The current model does not add artificial inheritance merely to demonstrate it.
Single responsibility	Services coordinate transactions; controllers handle HTTP; repositories persist; adapters call providers.

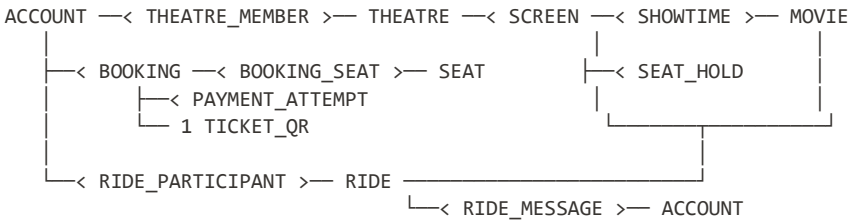
5. ER model and relationships

The main relationships are: an account may own multiple theatre memberships; a theatre contains multiple screens; a screen hosts showtimes; a movie appears in many showtimes; a customer creates bookings for a showtime; bookings contain seat snapshots and payment attempts; a confirmed booking has a ticket credential; and a customer posts or joins ride records linked to showtimes. Chat messages belong to a ride and an account. Constraints enforce uniqueness of venue/show identity, seat/show active allocation, user membership and provider idempotency keys.

Entity	Purpose
ACCOUNT	Identity, verified email/mobile, role/status
THEATRE	Jaipur venue, neighbourhood, address
THEATRE_MEMBER	Owner-to-theatre role mapping
SCREEN	Venue screen and seat layout
MOVIE	Title, language, certification, runtime
SHOWTIME	Movie + screen + start time + format
SEAT	Physical seat label and category
SEAT_HOLD	Temporary seat allocation and expiry
BOOKING	Customer, show, status, INR total, reference
BOOKING_SEAT	Booked seat and price snapshot
PAYMENT_ATTEMPT	Provider, idempotency key, status, amount
TICKET_QR	Opaque token hash, expiry, redemption status
RIDE	Driver, show, route, time, capacity, fare share
RIDE_PARTICIPANT	Ride membership/request state
RIDE_MESSAGE	Access-controlled chat content and timestamp

6. ER diagram

Simplified logical view; junction and operational tables are expanded in the entity list above.



7. Normalization

The target operational schema is designed to Third Normal Form (3NF). Each relation stores one kind of fact; attributes are atomic; primary keys identify rows; non-key attributes depend on the whole key and not on another non-key attribute. For example, movie metadata belongs to MOVIE instead of being copied into every show, while screen and theatre details are

stored once and linked by foreign keys. Theatre owner access is a junction relation rather than comma-separated owner IDs. Booking-seat rows store the seat and fare snapshot used for that transaction.

Why stop at 3NF for the transactional core: it removes the common insert/update/delete anomalies while keeping joins, constraints and transaction behavior understandable for a small booking system. Moving every relation to BCNF or 4NF can add junction tables and join cost without a demonstrated dependency or multivalued-fact problem. Normalize based on actual functional dependencies, not a higher normal-form label alone.

How to improve further: inspect real functional dependencies; decompose any relation whose determinant is not a candidate key (BCNF); isolate independent multivalued facts such as film languages or theatre amenities when they are stored as repeating values (4NF); use migrations and constraints to enforce the decompositions; and retain carefully selected read models/materialized views for performance. Denormalized reporting copies must remain derived, refreshable and outside the source-of-truth transaction model.

8. Security, safety and operational considerations

- Use provider-hosted payment pages or tokenization; never store card/UPI credentials. Verify webhook signatures, amount, currency and idempotency.
- Protect OTP with rate limits, expiry, attempt limits, abuse monitoring and secure account recovery. Keep provider secrets out of source control.
- Use opaque QR credentials and one-time atomic redemption. Do not place personal details in QR content.
- Enforce owner theatre scoping at the API and database query level, not only in the UI.
- Provide ride report/block tools, moderation logs and limited disclosure of contact details. Clearly state the platform is coordinating peer-to-peer cost sharing, not dispatching a taxi.
- Monitor payment settlement versus booking state; ticket message delivery retries; seat hold expiry; WebSocket fanout; and QR validation failures.

9. Current implementation status and next work

Available now	Required before real customer bookings
Clickable React prototype; theme switch; browsing and sample Jaipur venues; date/showtime selection; visual seat state and seeded price tiers; checkout method selection; OTP form; ride post form; owner-facing demo; Java domain entities and integration interfaces.	REST controllers and application services; PostgreSQL migrations; complete account/owner authorization; live seat inventory and seat holds; provider adapters; signed webhooks and refunds; QR generation/delivery/scanning; durable outbox; chat; audit/monitoring; deployment and support processes.

Implementation sequence: (1) agree on venue inventory and policies, (2) complete database migrations and identity, (3) build transactional seat holds plus concurrency handling, (4) integrate payment sandbox and idempotent webhook flow, (5) produce QR and delivery outbox, (6) add owner catalog tools and ride chat, (7) run security, concurrency, accessibility and pilot operations review.